



React.js Fast Track Notes

Topic	Key Points
Components	JS functions returning JSX
JSX	HTML in JS with rules
Props	Data passed like function args
Events	onClick, onChange, onSubmit, etc.
map/filter	Looping and transforming arrays
Routing	With react-router-dom
Lists	Use .map() with key prop
Styling	Inline via style={} or className
Hooks	Enabling State and Lifecycle Features in Functional Components useState, useReducer, useContext, useEffect
Axios	simplifies making promise-based HTTP requests to APIs with features like automatic JSON parsing,

31 React Components

- React components are JavaScript functions that return JSX markup.
- Component names **must start with a capital letter**.

```
function Welcome() {
  return <h1>Hello!</h1>;
}
export default Welcome;
```

Type	Function Style	Export Syntax	Import Syntax in App.js	Note
Named Export	Arrow Function	export const MyComponent = () => {}	import { MyComponent } from './...'	Use curly braces during import
Named Export	Regular Function	export function MyComponent() {}	import { MyComponent } from './...'	Use curly braces during import
Default Export	Arrow Function	const MyComponent = () => {}; export default MyComponent;	import MyComponent from './...'	Cannot write export default const ... directly
Default Export	Regular Function	export default function MyComponent() {}	import MyComponent from './...'	Cleanest for default export

JSX Basics

JSX lets you write HTML-like code inside JavaScript.

```
return (
  <>
    <h1>Hello</h1>
    <p>World</p>
  </>
)
```

- **Only one parent** element allowed in return()
Use **Fragments** (`<>...</>`) to avoid extra <div>
- Use JS inside {}: <h1>{name}'s List</h1>

Concept	Example
camelCase Attributes	className, onClick
Dynamic Data	{expression} inside JSX
Inline Styles	style={{ color: 'red' }}
JSX Comments	{/* This is a comment */}
Close tag	<img.../>, <input.../>, etc

Inline Styling in JSX

<h1 style={{ color: 'blue', backgroundColor: 'yellow' }}> Styled Text</h1>

```
function Avatar() {
  const pic = 'img.png';
  const css = { color: 'blue', backgroundColor: 'yellow' }
  return (
    <>
      <h1 style={css}> Hello </h1>
      <img className="pic" src={pic} />
    </>
  )
}
export default Avatar;
```

Image Handling

// **Public folder: Direct name**

// **src folder:**

First import img from './logo.png';

Then use

Props (Properties)

Pass data from parent to child components.

```
// App.js
<Product name="Laptop" price="70000" />

// Product.js
function Product(props) {
  return <h1>{props.name} : {props.price}</h1>;
}
```

Props as Objects:

```
// App.js
const info = { name: "ABC", dept: "CSE" };
<Component data={info} />

// Component.js
<h2>{props.data.name}</h2> // To get name
```

Props as Array:

```
// App.js
const students = ["A", "N", "R"];
<StudentList data={students} />

// Inside StudentList Component
<ul>
  { props.data.map((name, index) => (
    <li key={index}>{name}</li>
  ))}
</ul>
```

Event Handling

Event Type	Syntax	Example
onClick	onClick={handleClick}	alert("Hi!")
onChange	onChange={handleChange}	console.log(e.target.value)
onSubmit	onSubmit={handleSubmit}	e.preventDefault()
onDoubleClick	onDoubleClick={handler}	Alert on double click

onClick Example:**JS**

```
function handleClick() {  
  alert("Welcome!");  
}
```

JSX

```
<button onClick={handleClick}>Click</button>
```

//Pass Parameter:

```
//<button onClick={() => shoot("Goal!")}>Shoot</button>
```

onSubmit Example:**JS**

```
function hs(e) {  
  e.preventDefault();  
  alert("Submitted");  
}
```

JSX

```
<form onSubmit={hs}>  
  <input type="text" />  
  <button type="submit">Submit</button>  
</form>
```

onChange Example:**JS**

```
function handleChange(event) {  
  console.log (event.target.value);  
}
```

JSX

```
<input type="text" name="firstName" onChange={handleChange} />
```

Note: Avoid onClick={func()} unless intentional immediate call

map() and filter() in React

map() Example:

```
const arr = [1, 2, 3];  
arr.map(num => <h2>{num}</h2>);
```

map() with operation:

```
arr.map(n => <h2>{n} * 5 = {n * 5}</h2>)
```

filter() before map():

```
arr.filter(n => n > 3).map(n => <h2>{n}</h2>)
```

Lists and Keys

Each child in a list needs a **unique** `key` prop.

Keys help React track changes in lists.

```
students.map((s, i) => <li key={s.id}>{s.name}</li>)
```

Images in React

```
<img src={img1} alt="product" width="200" height="200" />
```

Display multiple images with map():

```
images.map((img, i) => <img key={i} src={img.src} alt="pic" />)
```

Combined map + props Example

```
const products = [  
  { name: "Pen", price: 10 },  
  { name: "Book", price: 50 }  
];  
  
products.map((item, i) => (  
  <Product key={i} name={item.name} price={item.price} />  
))
```

React Router (Page Navigation)

Install:

```
npm install react-router-dom
```

Basic Structure:

```
import { BrowserRouter as Router, Routes, Route, Link } from 'react-router-dom';

<Router>
  <nav>
    <Link to="/">Home</Link>
    <Link to="/shop">Shop</Link>
  </nav>
  <Routes>
    <Route path="/" element={ <Home /> } />
    <Route path="/shop" element={ <Shop /> } />
    <Route path="*" element={ <Nopage /> } />
  </Routes>
</Router>
```

404 Page:

```
<Route path="*" element={ <Nopage /> } />
```

Note: Construct Home.js, Shop.js etc as per requirement.

Combined map + props+Route Example

Main component

```
const products=[{name:"p1",price:20000},{name:"p2",price:14000}]
.....
<Route path="/product" element={ <Product info={products}/> } />
```

Product.js component

```
export default function Product(props){
  return(<
    { props.info.map((p)=>{
      return(
        <>
          <h1>{p.name}</h1>
          <h1>{p.price}</h1>
        </>
      )}})
  </>
}
```



React.js - (Hooks, Context, Forms, Effects, API)

Hooks Overview

Hooks let you use state and other React features without writing classes.

Rules:

✓ Call hooks at the top level (not inside loops/conditions).

✓ Use hooks only in React functions.

Hook	Purpose	Syntax
useState	Manage local state in function components	<code>const [state, setState] = useState(initialVal)</code>
useReducer	Complex state logic with reducer function	<code>const [state, dispatch] = useReducer(reducer, initialVal)</code>
createContext	Create global context	<code>const MyContext = createContext(defaultValue)</code>
useContext	Consume the nearest Context Provider's value	<code>const value = useContext(MyContext)</code>
useEffect	Perform side effects like API calls, timers, etc.	<code>useEffect(() => { ... }, [deps])</code>



useState Examples

Counter Example:

```
const [count, setCount] = useState(0);
```

.....

```
<button onClick={() => setCount(count + 1)}>Click</button>
```

Dynamic Text, Color, Show/Hide: Use multiple useStates to manage UI interactivity like toggle text, colors, and visibility. ([Refer Examples](#))

Form Handling with a Single useState:

```
const [data, setData] = useState({});  
setData({ ...data, [name]: value });
```

Basic Calculator: Convert number into integer

```
setResult(parseInt(data.num1) + parseInt(data.num2))
```

React Forms

Form Example with useState:

- Inputs: First name, Email, Password, City, Gender (radio), etc.
- Submit displays alert with form values.
- Controlled components: onChange updates state.

```
import { useState } from 'react'
function Myform2() {
  const[formdata,setformdata]=useState({});

  function handlechange(event){
    const {name,value} = event.target;
    setformdata({...formdata, [name]: value})
  }
  function handlesubmit(e) {
    e.preventDefault();
    alert("Your form has been submitted.\nName: " + formdata.fname)
  }

  return (
    <div>
      <form className="form-data" onSubmit={handlesubmit}>

        <label>First Name:</label>
        <input type="text" name="fname" onChange={handlechange} /><br/>

        <select onChange={handlechange} name='city'>
          <option value="Ahmedabad">Ahmedabad</option>
          <option value="Rajkot">Rajkot</option>
        </select>

        <input type="radio" name="gender" value="Male" onChange={handlechange} />Male
        <input type="radio" name="gender" value="Female" onChange={handlechange} />Female
        <button type="submit">Submit</button> <br/>
      </form>
    </div>
  )
}
export default Myform2
```

useReducer Hook

Syntax:

```
const [state, dispatch] = useReducer(reducerFunction, initialValue);
```

Increment Example:

```
function reducer(state, action) {  
  if (action.type === 'increment') return state + 1;  
}
```

Multi-action Reducer: use switch... case or If conditions

```
if (action.type === 'reset') return 0;
```

Button Dispatch Examples:

```
<button onClick={() => dispatch({ type: 'increment' })}>Increment</button>  
<button onClick={() => dispatch({ type: 'decrement' })}>Decrement</button>  
<button onClick={() => dispatch({ type: 'reset' })}>Reset</button>
```

useContext Hook

- **Step 1:** Create context

```
const MyContext = createContext();  
export {MyContext} also
```

- **Step 2:** Provide context

```
<MyContext.Provider value={value}>
```

- **Step 3:** Consume context

```
import {MyContext} from “./...”  
const val = useContext(MyContext);  
return(  
  <h1>{val}</h1>  
)
```

Pass First & Last Name:

- Main.js: provides context values
- UC2.js: consumes and displays full name

Nested Component Context Example:

- Context 1: style object
- Context 2: text like “Students”

P.jsx (Parent Component)

```
import React, { createContext } from "react";
import C1 from "./C1";

export const ColorContext = createContext();

function P() {
  const fontColor = "yellow";
  const bgColor = "blue";

  return (
    <ColorContext.Provider value={{ fontColor, bgColor }}>
      <C1 />
    </ColorContext.Provider>
  );
}
export default P;
```

C1.jsx (Child Component 1)

```
import React from "react";
import C2 from "./C2";

function C1() {
  return (
    <div>
      <C2 />
    </div>
  );
}
export default C1;
```

C2.jsx (Child Component 2)

```
import React, { useContext } from "react";
import { ColorContext } from "./P";

function C2() {
  const { fontColor, bgColor } = useContext(ColorContext);

  return (
    <div>
      <h1
        style={{
          color: fontColor,
          backgroundColor: bgColor,
```

```

        padding: "10px",
      }}
    >
    Full stack Development
  </h1>
</div>
);
}
export default C2;

```

useEffect Hook

When Does useEffect Run?

Dependency Array	When it runs
[]	Once on initial render only
[stateVar]	On first render + when stateVar changes
<i>No array</i>	On every render

Basic Example – Show Alert on Count Change:

```

const [count, setCount] = useState(0);
useEffect(() => {
  alert("Count updated: " + count);
}, [count]);

```

One-time Initialization Example: (Ignore react-StrictMode)

```

useEffect(() => {
  alert("Component mounted");
}, []);

```

Triggered Everytime when the page is rendered

```

useEffect(() => {
  alert("EveryTime");
});

```

API Integration with Axios

Install Axios:

```
npm install axios
```

Fetch Joke Example:

```
axios.get("https://official-joke-api.appspot.com/random_joke")
  .then(res => setJoke(res.data))
  .catch(err => console.error(err));
```

useEffect for Initial Fetch: fetchJoke is function which use to call API

```
useEffect(fetchJoke, []);
```

Ex.jsx

```
import React,{ useState,useEffect } from 'react'
import axios from "axios";
const Randomjokeapi = () =>
{
  const[joke,setJoke]=useState("");
  function fetchJoke() {
    axios
    .get("https://official-joke-api.appspot.com/random_joke")
    .then((response)=>{setJoke(response.data);})
    .catch((error)=>{console.error(error);})
  }
  useEffect(fetchJoke,[])
  return(
    <div>
      <h1>{joke.setup}</h1>
      <h3>{joke.punchline}</h3>
      <button onClick={ fetchJoke } >Generate Joke </button>
    </div>
  )
}
export default Randomjokeapi
```